Introduction to



with Application to Bioinformatics

- Day 4

Quiz: Review Day 3

Go to Canvas, `Modules -> Day 4 -> Review Day 3`

~20 minutes

1. What will this code output?

In [ ]:
```
row = 'sofa|2000|buy|Uppsala'
fields = row.split('|')
price = fields[1]
if price == 2000:
    print('The price is a number!')
if price == '2000':
    print('The price is a string!')
```

1. What will this code output?

In [ ]:
```
row = 'sofa|2000|buy|Uppsala'
fields = row.split('|')
price = fields[1]
if price == 2000:
    print('The price is a number!')
if price == '2000':
    print('The price is a string!')
```

- `split()` always gives a list of **strings**, so `price` is `'20`
- A string is never equal to a number: `'2000' == 2000` is
- Convert it if you need a number: `int(fields[1])`

2. What will this code output?

In [ ]:
```
print(sorted([2000, 30, 100]))
```

2. What will this code output?

In [ ]:
```
print(sorted([2000, 30, 100]))
```

Numbers are sorted by their **value**: from the smallest to the larges

3. What will this code output?

In [ ]:
```
print(sorted(['2000', '30', '100']))
```

3. What will this code output?

In [ ]:
```
print(sorted(['2000', '30', '100']))
```

- Strings are sorted **alphabetically**, one character at a time,
- The first characters are `'1'`, `'2'` and `'3'`, so the order
  `'30'`
- Same values, different type, different result!

4. How do you convert the string `'2000'`

- `int('2000')` or `float('2000')`

4. How do you convert the string '2000'

- int('2000') or float('2000')

In [ ]:
```
print(int('2000'), float('2000'))
```

4. How do you convert the string `'2000'`

- `int('2000')` or `float('2000')`

In [ ]:
```
print(int('2000'), float('2000'))
```

`int` gives a whole number, `float` gives a decimal number.

5. How do you convert the string `'0.5'`

- `float('0.5')`

5. How do you convert the string `'0.5'`

- `float('0.5')`

In [ ]:
```
float('0.5')
```

5. How do you convert the string `'0.5'`

- `float('0.5')`

In [ ]:
```
float('0.5')
```

`'0.5'` has decimals, so only `float` works: `int('0.5')` give

6. How do you get the real number from

- `float('1e9')`

6. How do you get the real number from

- `float('1e9')`

In [ ]:
```
float('1e9')
```

6. How do you get the real number from

- `float('1e9')`

In [ ]:
```
float('1e9')
```

`1e9` is scientific notation: $1 \times 10^9 = 1\,000\,000\,000$. `int()` does

6. How do you get the real number from

- `float('1e9')`

In [ ]:
```
float('1e9')
```

1e9 is scientific notation: $1 \times 10^9 = 1\,000\,000\,000$. `int()` does

In [ ]:
```
int('1e9')
```

7. What value will these conversions give

In [ ]:
```
values = [1, 0, '0', '', {}, [], [0]]
for x in values:
    print(repr(x), bool(x))
```

7. What value will these conversions give

In [ ]:
```
values = [1, 0, '0', '', {}, [], [0]]
for x in values:
    print(repr(x), bool(x))
```

- `0` , the empty string `''` and empty containers (`[]` , `{}`) a
- Everything else is `True` , even the string `'0'` : it is not emp

8. Container types: which type matches ea

- **list**: when order is important
- **dict**: to keep track of the relation between keys and values
- **set**: to check for membership. No order, no duplicates.

8. Container types: which type matches ea

- **list**: when order is important
- **dict**: to keep track of the relation between keys and values
- **set**: to check for membership. No order, no duplicates.

In [ ]:
```
genre_list = ["comedy", "drama", "drama", "sci-fi"
genre_list
```

8. Container types: which type matches e

- **list**: when order is important
- **dict**: to keep track of the relation between keys and values
- **set**: to check for membership. No order, no duplicates.

In [ ]:
```
genre_list = ["comedy", "drama", "drama", "sci-fi"
genre_list
```

In [ ]:
```
genres = set(genre_list)
genres
```

8. Container types: which type matches e

- **list**: when order is important
- **dict**: to keep track of the relation between keys and values
- **set**: to check for membership. No order, no duplicates.

In [ ]:
```
genre_list = ["comedy", "drama", "drama", "sci-fi"
genre_list
```

In [ ]:
```
genres = set(genre_list)
genres
```

In [ ]:
```
genre_counts = {"comedy": 1, "drama": 2, "sci-fi":
genre_counts
```

8. Container types: which type matches ea

- **list**: when order is important
- **dict**: to keep track of the relation between keys and values
- **set**: to check for membership. No order, no duplicates.

In [ ]:
```
genre_list = ["comedy", "drama", "drama", "sci-fi"
genre_list
```

In [ ]:
```
genres = set(genre_list)
genres
```

In [ ]:
```
genre_counts = {"comedy": 1, "drama": 2, "sci-fi":
genre_counts
```

The list keeps the order and the duplicates, the set removes the du
the dictionary links each genre to its count.

9. Match the following statements with th syntax

- `def` : starts a new function definition
- `return` : ends a function
- `print()` : shows the current state of a variable
- `#` : marks that a line should not be executed

9. Match the following statements with th syntax

- `def` : starts a new function definition
- `return` : ends a function
- `print()` : shows the current state of a variable
- `#` : marks that a line should not be executed

In [ ]:

```
def greet(message):  # def: starts a new function
    # print('message is:', message)
    if message == 'Hi':
        print('Hi!')
        return  # return: the function ends here
    print('This was not a Hi')

greet('Hi')
greet('Hello')
```

9. Match the following statements with th syntax

- `def` : starts a new function definition
- `return` : ends a function
- `print()` : shows the current state of a variable
- `#` : marks that a line should not be executed

In [ ]:
```
def greet(message):  # def: starts a new function
    # print('message is:', message)
    if message == 'Hi':
        print('Hi!')
        return  # return: the function ends here
    print('This was not a Hi')

greet('Hi')
greet('Hello')
```

- With `'Hi'`, `return` ends the function, so the last `print`
- The line starting with `#` is not executed: remove the `#` to s `message`

10. Which string method is used at line 2

- `.join()`

10. Which string method is used at line 2

- `.join()`

In [ ]:
```
x = ['h', 'e', 'l', 'l', 'o']
y = '_'.join(x)
print(y)
```

10. Which string method is used at line 2

- `.join()`

In [ ]:
```
x = ['h', 'e', 'l', 'l', 'o']
y = '_'.join(x)
print(y)
```

- `.join()` is a **string** method: the string `'_'` is put betwe
  list
- `.split()` does the opposite, from a string back to a list:

10. Which string method is used at line 2

- `.join()`

In [ ]:
```
x = ['h', 'e', 'l', 'l', 'o']
y = '_'.join(x)
print(y)
```

- `.join()` is a **string** method: the string `'_'` is put betwe
  list
- `.split()` does the opposite, from a string back to a list:

In [ ]:
```
y.split('_')
```

TODAY

Session 1

- More on functions:
    - return statement
    - positional arguments and keyword arguments
    - default and optional arguments

Session 2

- Comments and documentation
- Basic error handling
- User input in python

Session 3

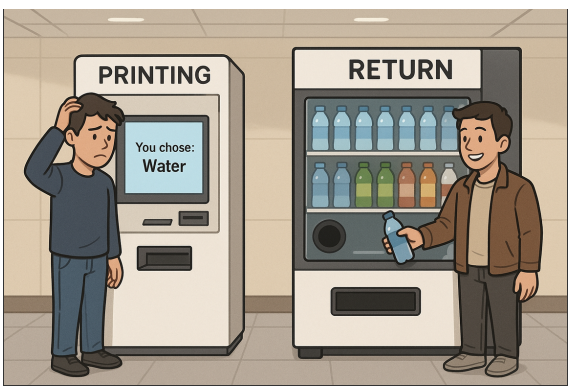
- Pandas - explore your data!

More on functions: `return` st

`return` vs `print`

Functions that `return` are easier to repurpose than those that p

Example



- **Print vending machine:** Only see an information. Cannot u
- **Return vending machine:** Have something real in your han
    - Drink it yourself (use the value)
    - Put it in your bag for later (store in a variable)
    - Give it to a friend (pass to another function)

Function with `return` statement

A function that counts the number of occurrences of `'C'` in the a

In [ ]:
```
def cytosine_count(nucleotides):
    count = 0
    for x in nucleotides:
        if x == 'c' or x == 'C':
            count += 1
    return count

count1 = cytosine_count('CATACTAC')
count2 = cytosine_count('tcgcag')
```

- Print the result

In [ ]:
```
print(f"{count1}\n{count2}")
```

- Repurpose the result: Sum the `'C'` occurrences

In [ ]:
```
total_count = count1 + count2
print(f"Total number of C: {total_count}")
```

Functions can return multiple values

In [ ]:

```
# Modify the function below for returning multiple
def cytosine_count(nucleotides):
    count = 0
    for x in nucleotides:
        if x == 'c' or x == 'C':
            count += 1
    return count
```

Functions without any `return` statement returns `None`

In [ ]:

```
def foo():
    do_nothing = 1

result = foo()
print(f'Return value of foo() = {result}')
```

Functions without any `return` statement returns `None`

In [ ]:
```
def foo():
    do_nothing = 1

result = foo()
print(f'Return value of foo() = {result}')
```

Return statement outcome

- Use `return` for all values that you might want to use later

simple words: **always**

Small detour: Python's value f values: `None`

- Default value for optional arguments
- Implicit return value of functions without a `return` statem
- `None` is `None`, not anything else (special value).

In [ ]:
```
None == 0
```

In [ ]:
```
None == False
```

In [ ]:
```
None == ''
```

In [ ]:
```
type(None)
```

More on functions: Keyword a

There are two types of arguments:

- **Positional:** values are passed in the same order as parame
  function
- **Keyword:** values are passed by parameters names while ca

Examples

In [ ]:

```
def myFavoriteMovie(pet_number, pet):
    print(f'I have {pet_number} {pet}')

# Positional arguments
myFavoriteMovie(2, "dogs")

# Keyword arguments
myFavoriteMovie(pet_number=3, pet="cats")
```

Why do we use keyword arguments?

- It increases the clarity and readability
- Independent on the order of the values
- Very helpfull when functions have many parameters

Why do we use keyword arguments?

- It increases the clarity and readability
- Independent on the order of the values
- Very helpfull when functions have many parameters

In [ ]:
```
record = 'gene_id INSR "insulin receptor"'

record.split(' ', 2)
```

Why do we use keyword arguments?

- It increases the clarity and readability
- Independent on the order of the values
- Very helpfull when functions have many parameters

In [ ]:
```
record = 'gene_id INSR "insulin receptor"'

record.split(' ', 2)
```

In [ ]:
```
record.split(sep=' ', maxsplit=2)
```

The order of keyword arguments does no

In [ ]:
```
record = 'gene_id INSR "insulin receptor"'

record.split(sep=' ', maxsplit=2)
```

The order of keyword arguments does no

In [ ]:
```
record = 'gene_id INSR "insulin receptor"'

record.split(sep=' ', maxsplit=2)
```

In [ ]:
```
record.split(maxsplit=2, sep=' ')
```

Both positional and keyword arguments it is allowed)

In [ ]:
```
record = 'gene_id INSR "insulin receptor"'

record.split(' ', maxsplit=2)
```

But there are some exceptions

In [ ]:
```
record = 'gene_id INSR "insulin receptor"'

record.split(sep=' ', 2)
```

But there are some exceptions

In [ ]:
```
record = 'gene_id INSR "insulin receptor"'

record.split(sep=' ', 2)
```

- Positional arguments must be **in front** of keyword argumen

Restrictions by purpose

General example

In [ ]:
```
def describe_gene(name, sequence, length):
    print(f"Gene: {name}, Sequence: {sequence}, Le

describe_gene('BRCA1', 'ATCG', 4)
describe_gene(name='BRCA1', sequence='ATCG', lengt
describe_gene('BRCA1', sequence='ATCG', length=4)
```

Restrictions by purpose

General example

In [ ]:
```
def describe_gene(name, sequence, length):
    print(f"Gene: {name}, Sequence: {sequence}, Le

describe_gene('BRCA1', 'ATCG', 4)
describe_gene(name='BRCA1', sequence='ATCG', lengt
describe_gene('BRCA1', sequence='ATCG', length=4)
```

Built-in function example

In [ ]:
```
sorted([1, 4, 100, 5, 6], reverse=True)
```

In [ ]:
```
sorted([1, 4, 100, 5, 6], True)
```

Restrictions by purpose

General example

In [ ]:
```
def describe_gene(name, sequence, length):
    print(f"Gene: {name}, Sequence: {sequence}, Le

describe_gene('BRCA1', 'ATCG', 4)
describe_gene(name='BRCA1', sequence='ATCG', lengt
describe_gene('BRCA1', sequence='ATCG', length=4)
```

Built-in function example

In [ ]:
```
sorted([1, 4, 100, 5, 6], reverse=True)
```

In [ ]:
```
sorted([1, 4, 100, 5, 6], True)
```

In [ ]:
```
sorted(iterable, /, *, key=None, reverse=False)
```

Restrictions by purpose

General example

In [ ]:
```
def describe_gene(name, sequence, length):
    print(f"Gene: {name}, Sequence: {sequence}, Le

describe_gene('BRCA1', 'ATCG', 4)
describe_gene(name='BRCA1', sequence='ATCG', lengt
describe_gene('BRCA1', sequence='ATCG', length=4)
```

Built-in function example

In [ ]:
```
sorted([1, 4, 100, 5, 6], reverse=True)
```

In [ ]:
```
sorted([1, 4, 100, 5, 6], True)
```

In [ ]:
```
sorted(iterable, /, *, key=None, reverse=False)
```

- arguments before `/` must be specified with position
- arguments after `*` must be specified with keyword

More on functions: Default an
arguments

How to define functions with default arg

In [ ]:
```
def format_sentence(subject, value = "finished", e
    return 'The ' + subject + ' is ' + value + end

format_sentence('lecture')
```

How to define functions with optional ar

- Convention: use the object `None`

In [ ]:
```
def format_sentence(subject, value, end='.', secon
    if second_value is None:
        return 'The ' + subject + ' is ' + value +
    else:
        return 'The ' + subject + ' is ' + value +

print(format_sentence('lecture', 'ongoing'))

#print(format_sentence('lecture', 'ongoing', secon
```

Summary

- `return` gives a value back, so you can save it in a variable
    - `print()` only shows a value on the screen, it do
    - a function can return several values at once: `retu`
    - a function without `return` gives back `None`
- `None` is Python's value for *nothing* or *missing*, and it is not or `''`
- Arguments can be given by position or by keyword: `open(` `mode='w')`
    - keyword arguments make the call easier to read, a matter
    - positional arguments must come **before** keyword
- Default arguments have a value already: `def format_se` `end='.')`, so you can skip them
- Optional arguments use `None` as default

Exercise 1 (~30-40 minutes)

- Notebook Day_4_Exercise_1
- Go to Canvas, `Modules -> Day 4 -> Exercise 1 –`
- Extra reading:
    - https://realpython.com/python-kwargs-and-args/
    - https://able.bio/rhett/python-functions-and-best-p

Documenting and commentin

It concerns two group of people:

- users: interested on how it is used
- developers: interested on how it works

Comments:

- mostly for developers. Describes the code
- pound sign (#) is used
- it is short
- tags can be used (BUG, TODO etc)

Documenting:

- mostly for users. Describes the use
- docstrings are used ("""..."""")
- it is quite long

Docstrings

In [ ]:

```
def process_file(filename, chrom, pos):
    """
    Read a very large vcf file, search for lines m
    chromosome chrom and position pos.

    Print the genotypes of the matching lines.
    """
    for line in open(filename):
        if not line.startswith('#'):
            col = line.split('\t')
            if col[0] == chrom and int(col[1]) ==
                print(col[9:])
```

Docstrings

In [ ]:
```
def process_file(filename, chrom, pos):
    """
    Read a very large vcf file, search for lines m
    chromosome chrom and position pos.

    Print the genotypes of the matching lines.
    """
    for line in open(filename):
        if not line.startswith('#'):
            col = line.split('\t')
            if col[0] == chrom and int(col[1]) ==
                print(col[9:])
```

In [ ]:
```
help(process_file)
```

Docstrings should always be in triple quotes even if they are short

In [ ]:
```
import random
def make_list(x):
    # Returns a random list of length x.
    li = list(range(x))
    random.shuffle(li)
    return li
```

In [ ]:
```
help(make_list)
```

What to write in a docstring

- What the function does
- The parameters (and their default values)
- What it returns

In [ ]:

```
def gc_content(sequence, decimals=2):
    """
    Calculate the GC content of a DNA sequence.

    Parameters:
        sequence: DNA sequence as a string, e.g. 'A
        decimals: number of decimals to round to (

    Returns:
        The fraction of G and C as a float, or Non
    """
    if len(sequence) == 0:
        return None
    gc = sequence.upper().count('G') + sequence.up
    return round(gc / len(sequence), decimals)
```

What to write in a docstring

- What the function does
- The parameters (and their default values)
- What it returns

In [ ]:
```
def gc_content(sequence, decimals=2):
    """
    Calculate the GC content of a DNA sequence.

    Parameters:
        sequence: DNA sequence as a string, e.g. 'A
        decimals: number of decimals to round to (

    Returns:
        The fraction of G and C as a float, or Non
    """
    if len(sequence) == 0:
        return None
    gc = sequence.upper().count('G') + sequence.up
    return round(gc / len(sequence), decimals)
```

In [ ]:
```
help(gc_content)
```

Comments

Use comments wherever the code is hard to understand

In [ ]:
```
my_list[5] += other_list[3]  # explain why you do
```

Comments: explain *why*, not *what*

The code already says **what** happens. The comment should say **w**

```
count += 1   # add 1 to count
```

Comments: explain *why*, not *what*

The code already says **what** happens. The comment should say **w**

```
count += 1   # add 1 to count
```

```
rating = float(fields[1].strip())  # ratings in th
spaces
```

A wrong comment is worse than no comment

```
# count the cytosines
count = sequence.count('G')
```

A wrong comment is worse than no comment

```
# count the cytosines
count = sequence.count('G')
```

- Which one is right, the comment or the code?
- When you change the code, update the comment too

Read more:

https://realpython.com/documenting-python-code/

comments: https://peps.python.org/pep-0008/#comments

docstrings: https://peps.python.org/pep-0257/

Error handling

Two paths for error handling:

1. Check conditions before performing a task

```
if x_can_be_done:
    do_this_with_x
else:
    error_handling
```

2. Perform the task and deal with errors afterwards

```
try:
    do_whatever_with_x
except <ErrorType>:
    error_handling
```

try and except: Check and report errors

Error handling

Two paths for error handling:

1. Check conditions before performing a task

```
if x_can_be_done:
    do_this_with_x
else:
    error_handling
```

2. Perform the task and deal with errors afterwards

```
try:
    do_whatever_with_x
except <ErrorType>:
    error_handling
```

try and except: Check and report errors

Reason for catching errors: Check if I can recover from this erro

Keep in mind: It is good to specify the error, e.g. `except Value`

Try and except basic example

In [ ]:
```
int("five")
```

Try and except basic example

In [ ]:
```
int("five")
```

Your own message + Python's message

In [ ]:
```
try:
    int("five")
except ValueError:
    print("Error: try again")
```

Catching a Recoverable Error

Scenario: I am perfect

Input:

- My code is perfect
- The data I use is not perfect (e.g. some entries are invalid o

Output:

- The code will throw an error and stop, making it impossible
  data

In [ ]:
```
with open('../files/bank_statement.txt', 'r', enco
    content = bank_file.read()
    print(content)
```

In [ ]:

```
fh = open("../files/bank_statement.txt", "r", enco

total = 0

for line in fh:
    expenses = line.strip().split()
    store = expenses[0]
    price = float(expenses[1])
    if store == 'ICA':
        total = total + price

fh.close()
print('Total amount spent on ICA is: '+str(total))
```

In [ ]:
```
fh = open("../files/bank_statement.txt", "r", enco

total = 0

for line in fh:
    expenses = line.strip().split()
    store = expenses[0]
    price = float(expenses[1])
    if store == 'ICA':
        total = total + price

fh.close()
print('Total amount spent on ICA is: '+str(total))
```

Two main problems:

- The code is exiting but there is nothing wrong with it
- The file is not closed

In [ ]:
```
print(f"File closed after error? {fh.closed}")
```

Raise your own error

Scenario: Validating a variable

- At some point in my code, I want to stop execution if a varia
- How can I interrupt the execution and raise an error?

In [ ]:
```
# Accept only integers
my_number=3.2

if type(my_number) is not int:
    print("Oooooohhhhh NOOOOOOOO")

print("more code follows")
```

User input

The code asks the user for input

- `input()`

In [ ]:
```
give_input = input("Give a number: ")

print(f'You pressed the number {give_input}')
```

Multiple inputs

In [ ]:
```
s1=input("s1: ")
s2=input("s2: ")
s3=input("s3: ")
s4=input("s4: ")
students_list = [s1, s2, s3, s4]
students_list
```

Type of input

In [ ]:
```
number_in=input("Give me a number: ")

type(number_in)
```

Type of input

In [ ]:
```
number_in=input("Give me a number: ")

type(number_in)
```

The type of the input is **ALWAYS** string

Ensure correct type of input (exercise)

We want a script which asks the user to give an integer number. If
not integer then inform about that and ask again about an integer

- Think about it and we will do it together
- Hint: inputs have string type

In [ ]:

Summary

- Docstrings (`"""..."""`) are for **users**: what the function
  what it returns
    - read them with `help()`
- Comments (`#`) are for **developers**: explain *why*, not *what*
    - when you change the code, update the comment
- `try..except` catches an error, so your program does no
    - name the error you expect: `except ValueErro`
    - `except ValueError as err:` gives you Pyth
- `with open(...)` closes the file for you, even when an e
- `raise` stops the program with your own error: `raise Ty`
  `number is not integer")`
- `input()` asks the user for a value, and it **always** gives a s
    - convert it with `int()` or `float()`

Exercise 2 (~20-30 minutes)

- Notebook Day_4_Exercise_2
- Go to Canvas, `Modules -> Day 4 -> Exercise 2 -`

Lunch

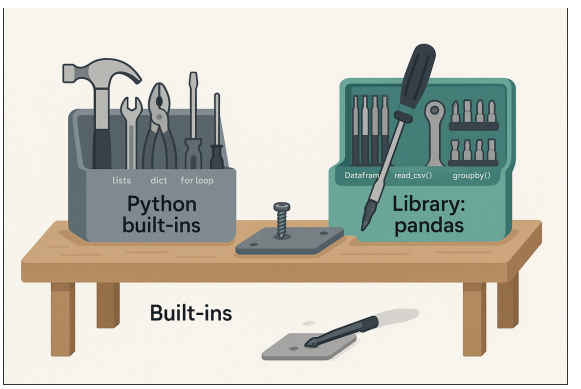
Pandas!!!



Pandas

- Library for working with tabular data
- Data analysis:
    - filter
    - transform
    - aggregate
    - plot
- Main hero: the `DataFrame` type

Built-in vs library tools - 1



Built-in vs library tools - 2

Task of the script:

- Load movies data from a file
- Calculate the average rating of all movies
- Filter the great movies: Rating >= 8.5
- Display them sorted by Rating (highest first), showing Title,

Built-in solution

In [ ]:

```
# 1. LOAD DATA
movies = []
for line in open('../downloads/250.imdb', encoding=
    if line.startswith('#'):  # skip the header li
        continue
    fields = line.strip().split('|')
    rating = float(fields[1])
    year = int(fields[2])
    title = fields[6]
    movies.append([title, year, rating])

print(f"Loaded {len(movies)} movies")

# 2. AVERAGE RATING
total = 0
for movie in movies:
```

```
    total = total + movie[2]

average = total / len(movies)
print(f"\nAverage Rating: {average:.2f}")

# 3. FILTER Rating >= 8.5
top_movies = []
for movie in movies:
    if movie[2] >= 8.5:
        top_movies.append(movie)

# 4. SORT by Rating, highest first
def get_rating(movie):
    return movie[2]

top_movies.sort(key=get_rating, reverse=True)

print(f"\nGreat Movies (Rating >= 8.5): {len(top_m
print(f"{'Title':<52} {'Year':>5} {'Rating':>7}")
print("-" * 66)
for movie in top_movies:
    print(f"{movie[0]:<52} {movie[1]:>5} {movie[2]
```

Library (pandas) solution

In [ ]:

```
import pandas as pd

# 1. LOAD DATA
df = pd.read_csv('../downloads/250.imdb', sep='|')

# Strip spaces from ALL column names at once
df.columns = df.columns.str.strip()

# Rename '# Votes' to 'Votes'
df = df.rename(columns={'# Votes': 'Votes'})

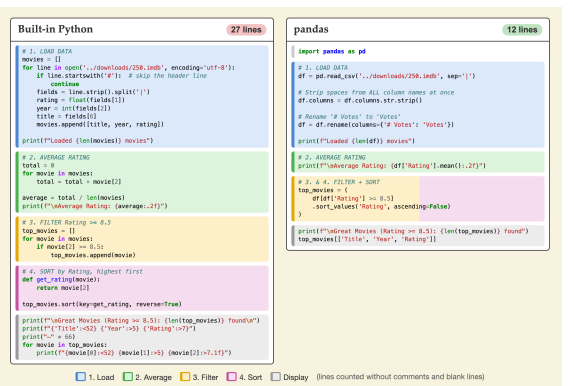
print(f"Loaded {len(df)} movies")

# 2. AVERAGE RATING
print(f"\nAverage Rating: {df['Rating'].mean():.2f

# 3. & 4. FILTER + SORT
top_movies = (
    df[df['Rating'] >= 8.5]
    .sort_values('Rating', ascending=False)
)

print(f"\nGreat Movies (Rating >= 8.5): {len(top_m
top_movies[['Title', 'Year', 'Rating']]
```

Built-in vs library tools - 3

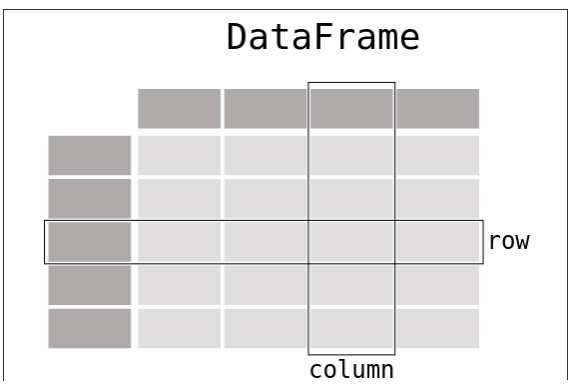


Benefits of shorter code

- Easier to read: you see **what** is done, not **how**
- Fewer lines, fewer places for bugs
- Easier to modify it: to sort by year, change `'Rating'` to

Pandas DataFrame

Pandas Dataframe is a **2D data structure**. In other words, is a tab
columns.



Creating a small DataFrame

In [ ]:
```
data = {
    'age': [1,2,3,4],
    'circumference': [2,3,5,10],
    'height': [30, 35, 40, 50]
}
```

In [ ]:
```
# Locate row
```

In [ ]:
```
# add name index
row_index = ["tree1", "tree2", "tree3", "tree4"]
```

In [ ]:
```
# Locate row
```

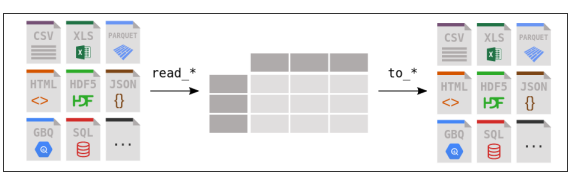
In [ ]:
```
# drop index
df = df.reset_index(drop=True)
df
```

Pandas can import data from many forma

- `pd.read_table` : tab separated values `.tsv`
- `pd.read_csv` : comma separated values `.csv`
- `pd.read_excel` : Excel spreadsheets `.xlsx`

DataFrame can export data to many forma

- `df.to_csv(sep='\t')` : tab separated values `.tsv`
- `df.to_csv()` : comma separated values `.csv`
- `df.to_excel()` : Excel spreadsheets `.xlsx`


CSV
XLS
PARQUET
HTML
HDF5
JSON
GBQ
SQL
...
read_*
to_*
CSV
XLS
PARQUET
HTML
HDF5
JSON
GBQ
SQL
...

Orange tree data

In [ ]:
```
df = pd.read_table('../downloads/Orange_1.tsv')
df
```

Read larger file

In [ ]:
```
df_big = pd.read_csv('../downloads/Orange_1_big.cs
df_big
```

Read larger file

In [ ]:
```
df_big = pd.read_csv('../downloads/Orange_1_big.cs
df_big
```

Do not panic

In [ ]:
```
pd.options.display.max_rows
```

Overview of your data, basic statistics

In [ ]:
```
df
```

Overview of your data, basic statistics

In [ ]:
```
df
```

In [ ]:
```
df.shape
```

Overview of your data, basic statistics

In [ ]:
```
df
```

In [ ]:
```
df.shape
```

In [ ]:
```
df.describe()
```

Overview of your data, basic statistics

In [ ]:
```
df
```

In [ ]:
```
df.shape
```

In [ ]:
```
df.describe()
```

In [ ]:
```
df.max()
```

Overview of your data, basic statistics

In [ ]:
```
df
```

In [ ]:
```
df.shape
```

In [ ]:
```
df.describe()
```

In [ ]:
```
df.max()
```

In [ ]:
```
df_big.head(10)
```

Overview of your data, basic statistics

In [ ]:
```
df
```

In [ ]:
```
df.shape
```

In [ ]:
```
df.describe()
```

In [ ]:
```
df.max()
```

In [ ]:
```
df_big.head(10)
```

In [ ]:
```
df_big.tail()
```

Overview of your data, basic statistics

In [ ]:
```
df
```

In [ ]:
```
df.shape
```

In [ ]:
```
df.describe()
```

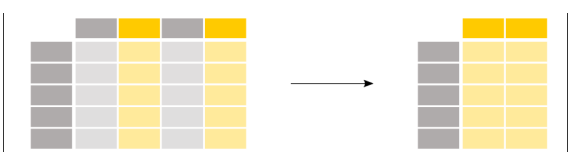
In [ ]:
```
df.max()
```

In [ ]:
```
df_big.head(10)
```

In [ ]:
```
df_big.tail()
```

In [ ]:
```
df_big.info()
```

Selecting columns, rows and cells from a



Selecting one column

```
dataframe.columnname
dataframe['columnname']
```

Selecting one column

```
dataframe.columnname
dataframe['columnname']
```

In [ ]:
```
df
```

In [ ]:
```
df.age
```

In [ ]:
```
df['age']
```

Selecting one column

```
dataframe.columnname
dataframe['columnname']
```

In [ ]:
```
df
```

In [ ]:
```
df.age
```

In [ ]:
```
df['age']
```

`df.age` only works when the column name has no spaces and is
method.

For example, a column called *count* returns the `.count()` metho

So the **advice** is: `df['age']` always works, so prefer it.

Unique values of a column

In [ ]:
```
df['Tree'].unique()
```

Unique values of a column

In [ ]:
```
df['Tree'].unique()
```

How many of each?

In [ ]:
```
df['Tree'].value_counts()
```

Unique values of a column

In [ ]:
```
df['Tree'].unique()
```

How many of each?

In [ ]:
```
df['Tree'].value_counts()
```

- `unique()` gives the different values and `value_counts` each one appears.
- Bioinformatics examples: variants per chromosome, reads p biotype.

Be careful with the headers of the column

In [ ]:

```
import pandas as pd
data = {
    'age ': [1,2,3,4],
    'circumference': [2,3,5,10],
    'height': [30, 35, 40, 50]
}
df_test = pd.DataFrame(data)
df_test
```

Be careful with the headers of the column

In [ ]:
```
import pandas as pd
data = {
    'age ': [1,2,3,4],
    'circumference': [2,3,5,10],
    'height': [30, 35, 40, 50]
}
df_test = pd.DataFrame(data)
df_test
```

In [ ]:
```
df_test.age
```

Be careful with the headers of the column

In [ ]:
```
import pandas as pd
data = {
    'age ': [1,2,3,4],
    'circumference': [2,3,5,10],
    'height': [30, 35, 40, 50]
}
df_test = pd.DataFrame(data)
df_test
```

In [ ]:
```
df_test.age
```

In [ ]:
```
df_test['age']
```

Strip the spaces

In [ ]:

Selecting multiple columns

In [ ]:
```
df
```

In [ ]:
```
df['age']
```

Selecting rows from a dataframe

In [ ]:
```
df
```

In [ ]:
```
df.loc[0] # select the first row
```

In [ ]:
```
df.loc[1:3] # select from row 2 to 4
```

In [ ]:
```
df.loc[1, 3] # select row 2 and 4
```

Selecting rows from a dataframe

In [ ]:
```
df
```

In [ ]:
```
df.loc[0] # select the first row
```

In [ ]:
```
df.loc[1:3] # select from row 2 to 4
```

In [ ]:
```
df.loc[1, 3] # select row 2 and 4
```

REMINDER:

- `.loc` uses the index labels, not the position of the row
- when range is used in `.loc` the *stop* part is included

Selecting cells from a dataframe

In [ ]:
```
df
```

In [ ]:
```
# Single cell
df.loc[0, 'age']

# One row, multiple columns

# Multiple rows, one column

# Multiple rows and columns
```

Run statistics on specific rows, columns, cells

In [ ]:
```
df[['age', 'circumference']].describe()
```

In [ ]:
```
df['age'].std()
```

In [ ]:
```
df.loc[0].describe()
```

In [ ]:
```
df.loc[0, 'age'] - df['age'].mean()
```

Selecting data from a dataframe by positi

```
dataframe.iloc[row_position]
dataframe.iloc[row_position, column_position]
dataframe.iloc[start:stop]
```

Note: the *stop* part in the `.iloc` is not included (different from

Further reading from pandas documentation: https://pandas.pydat
pandas.DataFrame.iloc.html

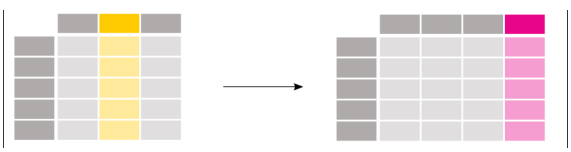
In [ ]:
```
df
```

In [ ]:
```
# Show the first column

# Show the second row

# Show the cell of the second row and the first co
```

Creating new column derived from existi



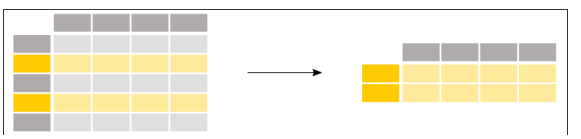
In [ ]:
```
import math
df['radius'] = df['circumference'] / (2.0 * math.p
df
```

Quiz time

Go to Canvas, `Modules -> Day 4 -> PyQuiz 4.1`

~5 min

Pandas concatenate



In [ ]:
```
df1 = pd.DataFrame({
    'age': [1,2,3,4],
    'circumference': [2,3,5,10],
    'height': [30, 35, 40, 50]
})

df1
```

In [ ]:
```
df2 = pd.DataFrame({
    'name': ['palm', 'ada', 'ek', 'olive'],
    'price': [1423, 2000, 102, 30]
})

df2
```

In [ ]:
```
df_con = pd.concat([df1,df2])
df_con
```

In [ ]:
```
# index is not reset and it will cause some proble
df_con.loc[0]
```

Reset index in concatenated dataframe

In [ ]:
```
df_con=df_con.reset_index(drop=True)
df_con
```

Control the concatenation axis:

- axis=0 (default): rows concatenation
- axis=1: columns concatenation

In [ ]:
```
pd.concat([df2, df1], axis=0).reset_index(drop=Tru
```

In [ ]:
```
pd.concat([df2, df1], axis=1)
```

Selecting/filtering the dataframe by cond

e.g.

- Only trees with age larger than 100
- Only tree with circumference shorter than 20

In [ ]:
```
df = pd.read_table('../downloads/Orange.tsv')
df
```

Selecting with condition

`dataframe[`*`condition`*`]`

Selecting with condition

`dataframe[`*`condition`*`]`

In [ ]:
```
df[df['Tree'] == 1]
```

Selecting with condition

`dataframe[condition]`

In [ ]:
```
df[df['Tree'] == 1]
```

In [ ]:
```
df[df['age'] > 500]
```

Filter with multiple conditions

```
dataframe[(condition 1) & (condition 2)]

dataframe[(condition 1) | (condition 2)]
```

Filter with multiple conditions

```
dataframe[(condition 1) & (condition 2)]
```

```
dataframe[(condition 1) | (condition 2)]
```

In [ ]:
```
df[(df['age'] > 100) & (df['age'] <= 250)]
```

Filter with multiple conditions

```
dataframe[(condition 1) & (condition 2)]
```

```
dataframe[(condition 1) | (condition 2)]
```

In [ ]:
```
df[(df['age'] > 100) & (df['age'] <= 250)]
```

In [ ]:
```
df[(df['age'] > 500) & (df['circumference'] < 100)]
```

Filter with multiple conditions

```
dataframe[(condition 1) & (condition 2)]
```

```
dataframe[(condition 1) | (condition 2)]
```

In [ ]:
```
df[(df['age'] > 100) & (df['age'] <= 250)]
```

In [ ]:
```
df[(df['age'] > 500) & (df['circumference'] < 100)]
```

In [ ]:
```
df[(df['Tree'] == 1) | (df['Tree'] == 3)]
```

Filter with multiple conditions

```
dataframe[(condition 1) & (condition 2)]
```

```
dataframe[(condition 1) | (condition 2)]
```

In [ ]:
```
df[(df['age'] > 100) & (df['age'] <= 250)]
```

In [ ]:
```
df[(df['age'] > 500) & (df['circumference'] < 100)]
```

In [ ]:
```
df[(df['Tree'] == 1) | (df['Tree'] == 3)]
```

- & means "and": both conditions must be true
- | means "or": at least one condition must be true
- Put each condition in brackets ( )
- Do not use the Python words **and/or** here

Small exercise 1

- Find the maximal circumference and then filter the datafram

In [ ]:
```
df
```

Statistics per group

Question: what is the maximum circumference of each tree?

In [ ]:

Statistics per group

Question: what is the maximum circumference of each tree?

In [ ]:

- `groupby('Tree')` : split the table into one group per tre
- `['circumference']` : choose the column
- `.max()` : calculate the maximum for each group; the result table

Bioinformatics examples: the mean expression per gene, the nu sample.

Saving your results to a file

In [ ]:
```
# create table
old_trees = df[df['age'] > 1000]
```

In [ ]:
```
# read table
```

Quiz time

Go to Canvas, `Modules -> Day 4 -> PyQuiz 4.2`

Plotting

- pandas can plot your data directly
- One pattern for all plots:

```
df.plot(x='column', y='column', kind='...')
```

- More information: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.plot.html

Which plot for which question?

`kind`	Use it for	Bioinforma
`line`	change over time or in order	growth, tim
`scatter`	relationship between two numbers	expression
`hist`	distribution of one variable	read length
`box`	comparing distributions between groups	expression
`bar`	comparing categories	number of chromosom

The data we will plot

In [ ]:
```
import pandas as pd

df = pd.read_table('../downloads/Orange.tsv')
df_big = pd.read_csv('../downloads/Orange_1_big.cs
```

Line plot

In [ ]:

```
tree1 = df[df['Tree'] == 1]
tree1
# Simplest plot: df.columnname.plot() - default=li


# Nicer plot: set x and y axis, kind, title, label
```

Scatter plot

In [ ]:
```
df.plot(kind='scatter', x='age', y='circumference'
        title='Age vs circumference',
        xlabel='Age (days)', ylabel='Circumference
```

Histogram

In [ ]:
```
df_big.plot(kind='hist', y='height', bins=15,
            title='Distribution of tree height', x
```

Box plot

In [ ]:
```
df.boxplot(column='circumference', by='Tree')
```

Bar plot

In [ ]:
```
df.groupby('Tree')['circumference'].max().plot(kin
        title='Maximum circumference per tree',
        xlabel='Tree', ylabel='Circumference (mm)'
```

Size and font

In [ ]:
```
# figsize and fontsize
df.plot(kind='scatter', x='age', y='circumference'
        title='Age vs circumference',
        xlabel='Age (days)', ylabel='Circumference
        fontsize=16, figsize=(12, 8))
```

Build your plot step by step

In [ ]:
```
df.plot(kind='scatter', x='age', y='circumference'
        figsize=(12, 8), fontsize=16)
```

Saving a plot

In [ ]:
```
ax = df.plot(kind='scatter', x='age', y='circumfer
             title='Age vs circumference',
             xlabel='Age (days)', ylabel='Circumfe

ax.figure.savefig('orange_scatter.png', dpi=300)
```

Exercise 3 (~30 minutes)

- Go to Canvas, `Modules -> Day 4 -> Exercise 3 –`
- Realistic data: RNA-seq results in an **Excel** file, cells in norm
- **Everyone**: Parts 1 and 2
    - Explore the samples and the results with pandas
    - Make bar plots and a histogram
- **Want more?** Part 3
    - Volcano plot, your own function, MA plot
- Stuck? Use `Day_4_exercise_3_hints.ipynb`